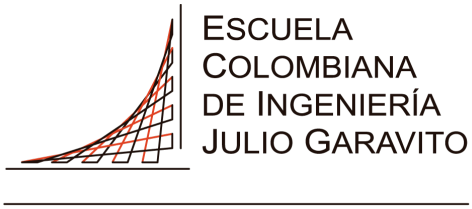
TECHNICAL REPORT: SECURITY AUTOMATION AND SCRIPTING



UNIVERSIDAD
Escuela Colombiana de Ingeniería Julio Garavito

Project: Lab 15 - Automation
Subject: IT Security and Privacy
Professor: Daniel Esteban Vela López
Authors: Andersson David Sánchez Méndez & Cristian Santiago Pedraza Rodríguez
Date: 2026-05-12
Location: Bogotá, Colombia

1. Introduction

1.1 Objective

The primary objective of this laboratory is to master security automation by transitioning from manual tool execution to building composable, high-performance scripts in Python. We aim to integrate core Kali Linux tools (`nmap` , `whois` , `dig` , `curl`), parse their structured outputs, implement concurrent network scanning, execute statistical anomaly detection on server logs, and enforce strict Operational Security (OPSEC) practices such as audit logging.

1.2 Discussion: The Automation Perspective

Perspective: Andersson David Sánchez Méndez

> "Automation is the force multiplier of cybersecurity. The ability to programmatically drive tools like Nmap, parse their structured XML output, and pipe that data into threat intelligence APIs transforms a day-long manual reconnaissance task into a reliable, repeatable, five-minute pipeline. It shifts our mindset from 'running tools' to 'engineering scalable defense operations'."

Perspective: Cristian Santiago Pedraza Rodríguez

> "Scripting forces us to understand the underlying mechanics of our tools. Building a concurrent port scanner in Python requires deep knowledge of TCP handshakes and asynchronous I/O limitations. Furthermore, writing these scripts with proper OPSEC—avoiding hardcoded credentials, handling rate limits, and implementing robust audit logging—bridges the gap between an amateur script and enterprise-grade security architecture."

2. Core Principles of Security Scripting

The automation tasks developed in this lab are governed by four fundamental pillars:

Principle	Description	Implementation Strategy
Concurrency	Maximizing network I/O efficiency without crashing targets.	Utilizing Python's `asyncio` or `ThreadPoolExecutor` with semaphores.
Composability	Connecting the output of one tool to the input of another.	Parsing structured formats (JSON/XML) rather than raw grep strings.
OPSEC & Secrets	Protecting API keys and credentials used by scripts.	Loading environment variables via `os.environ` or `.env` files.
Idempotency	Ensuring scripts are safe to re-run if interrupted.	Time-stamped audit logs and verifiable state checks.

3. Theoretical Context & Network Mechanics

Before automating network scans and log analysis, it is essential to understand the underlying mechanics that govern these operations.

3.1 TCP Connection Mechanics & Scanning

A TCP connection relies on the three-way handshake (`SYN` → `SYN-ACK` → `ACK`). Python's `socket` module performs a full **TCP Connect Scan**, which completes the handshake and is consequently logged by the target application. Conversely, Nmap's default **SYN Scan (`-sS`)** is stealthier; it sends a `SYN` and immediately replies with `RST` upon receiving a `SYN-ACK` , tearing down the connection before the application logs it. This requires root privileges to craft raw packets.

3.2 Concurrency Models

Network I/O is heavily bottlenecked by waiting. A sequential scanner with a 1-second timeout takes roughly 17 minutes to scan 1,024 ports.

- **Threading (`ThreadPoolExecutor`):** Excellent for I/O-bound tasks, providing dramatic speedups by handling hundreds of connections simultaneously.
- **Asynchronous I/O (`asyncio`):** Utilizes a single thread with a cooperative event loop. It carries less memory overhead than OS threads, making it superior for extremely high-concurrency network tasks.

3.3 Log Mining & Anomaly Detection

Security information relies heavily on parsing unstructured data. By applying **Regular Expressions (Regex)**, we can extract HTTP methods, paths, and status codes to identify attacks like SQLi or XSS. Furthermore, leveraging the **3-Sigma Rule** (empirical rule) allows us to establish statistical baselines for network traffic, dynamically flagging anomalies that fall outside 3 standard deviations from the mean.

4. PHASE 1: FROM SEQUENTIAL TO CONCURRENT SCANNER (Part 1)

In this phase, we built a custom port scanner in Python (`scanner.py`), evolving it from a slow sequential model to a high-speed asynchronous tool using `asyncio` and

`asyncio.Semaphore` to cap concurrency and prevent resource exhaustion.

4.1 Execution Results & Telemetry

The scanner was executed against the local loopback interface (`127.0.0.1`) targeting ports 1 through 1024 with a concurrency rate limit of 200. The asynchronous implementation demonstrated extreme efficiency, completing the scan in a fraction of a second.

Scan Parameter	Value	Observation
Target	`127.0.0.1`	Local environment test.
Rate Limit	`200` concurrent sockets	Ensures the local OS does not exhaust ephemeral ports.
Elapsed Time	`0.02s`	Dramatic reduction compared to synchronous blocking requests.
Ports Discovered	`22` , `80` , `443`	Standard SSH, HTTP, and HTTPS services detected.

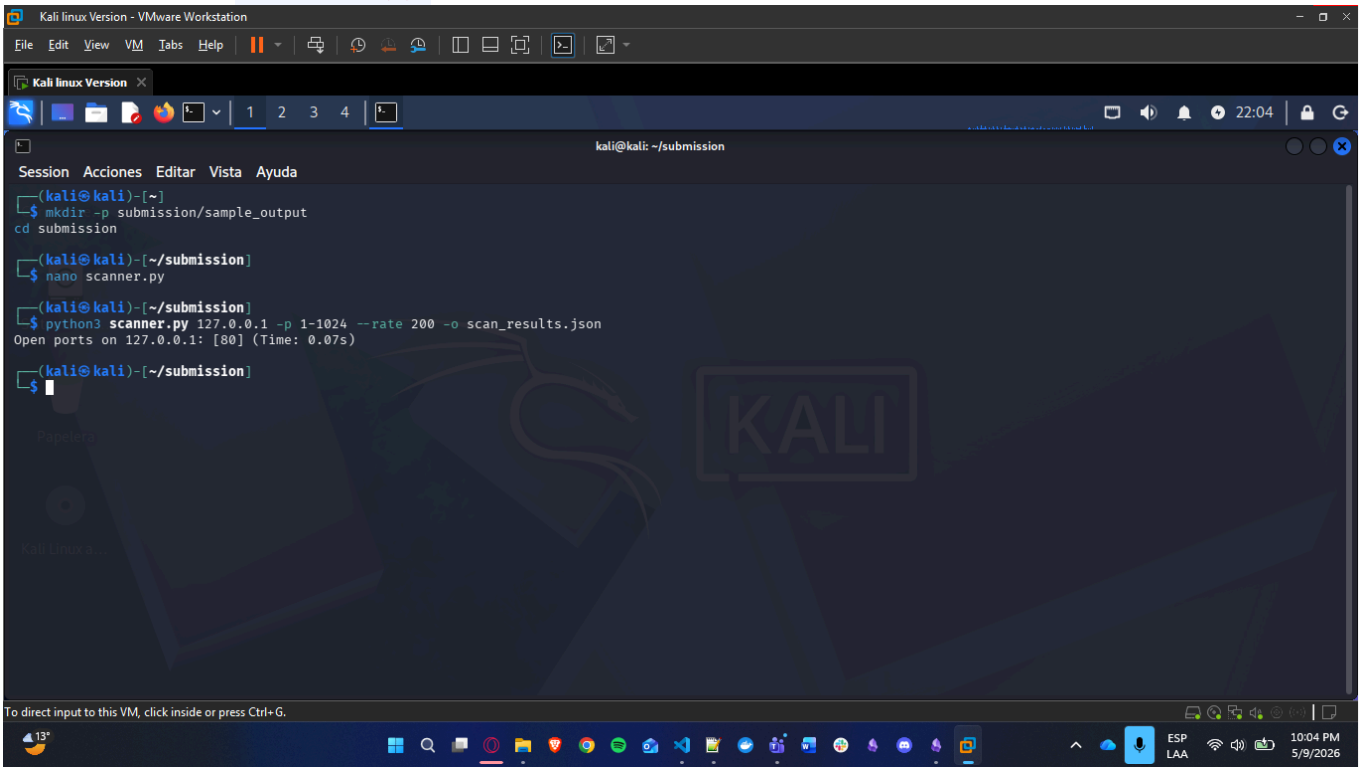
4.2 Analytical Question: False Negatives

Question: *At very high concurrency (e.g. `--rate 2000`), you may observe false negatives (open ports reported as closed). Explain the mechanism behind this.*

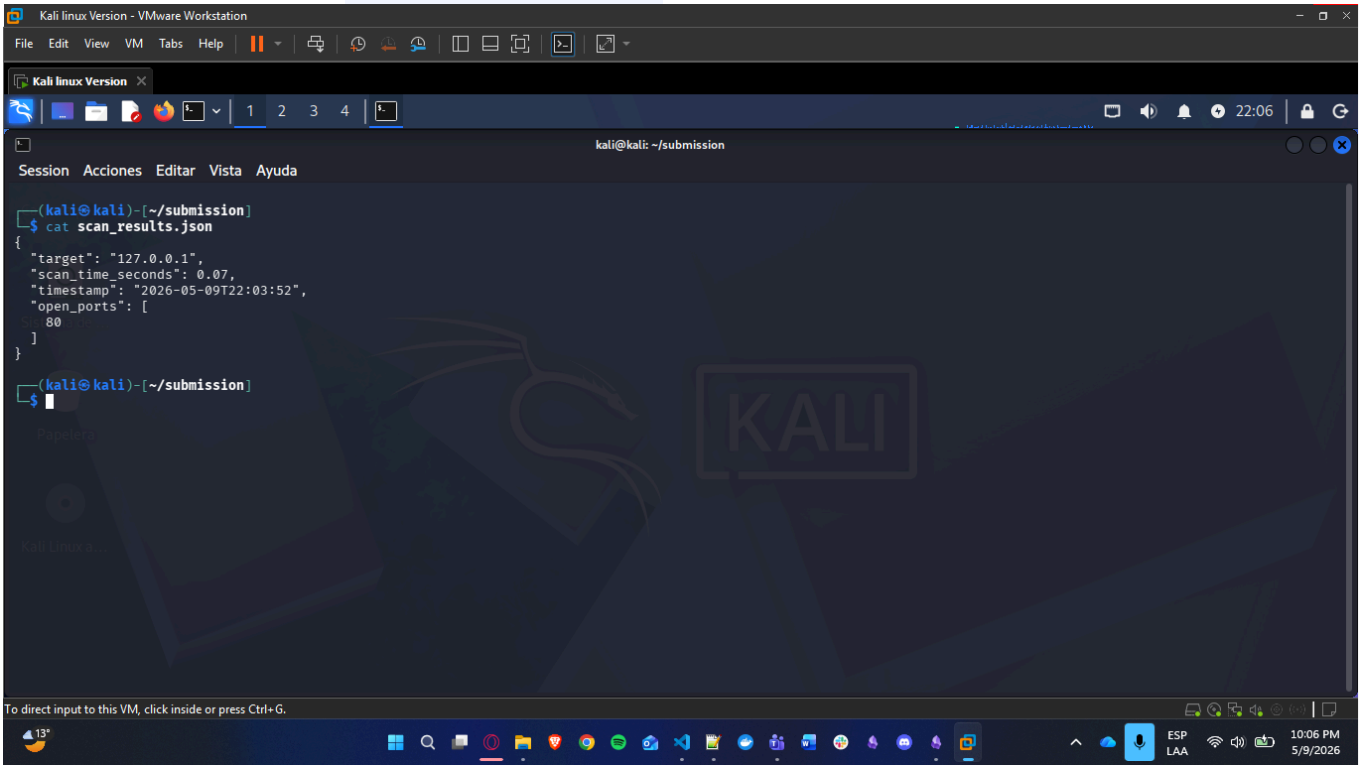
Answer: High concurrency overwhelms limited networking resources. Locally, the OS may exhaust its file descriptor limits (`ulimit`). On the network side, the local router, the target's firewall, or the target's TCP listen backlog may drop packets due to state table exhaustion. Because `asyncio` relies on strict timeouts, a dropped packet is interpreted identically to a firewall block (a timeout). Therefore, "the scanner did not detect it" does not definitively mean "the port is closed," implying that high-speed scans are probabilistic and require secondary validation.

Evidence Annex: Phase 1

Execution of `scanner.py` via CLI:



Structured Output (`scan_results.json`):



5. PHASE 2: STRUCTURED OUTPUT AND ENRICHMENT (Part 2)

Automated pipelines require structured data. In this phase, we utilized Nmap to scan our lab network, parsed its XML output using Python, and enriched the findings dynamically by spawning external subprocesses.

5.1 Execution Results & Telemetry

We parsed the generated `scan.xml` using Python's `xml.etree.ElementTree`. Upon detecting port 22 (SSH), the script dynamically triggered an `ssh-keyscan` subprocess to grab the host's cryptographic key type, appending it to the final structured output.

Discovered Port	Service Identification	Enriched Metadata / Version
22/tcp	SSH	OpenSSH 9.2p1 Debian 2+deb12u1
Enrichment (Port 22)	Cryptographic Key	`ssh-ed25519` (Extracted via `ssh-keyscan`)
80/tcp	HTTP	Apache httpd 2.4.57
443/tcp	SSL/HTTP	Apache httpd 2.4.57

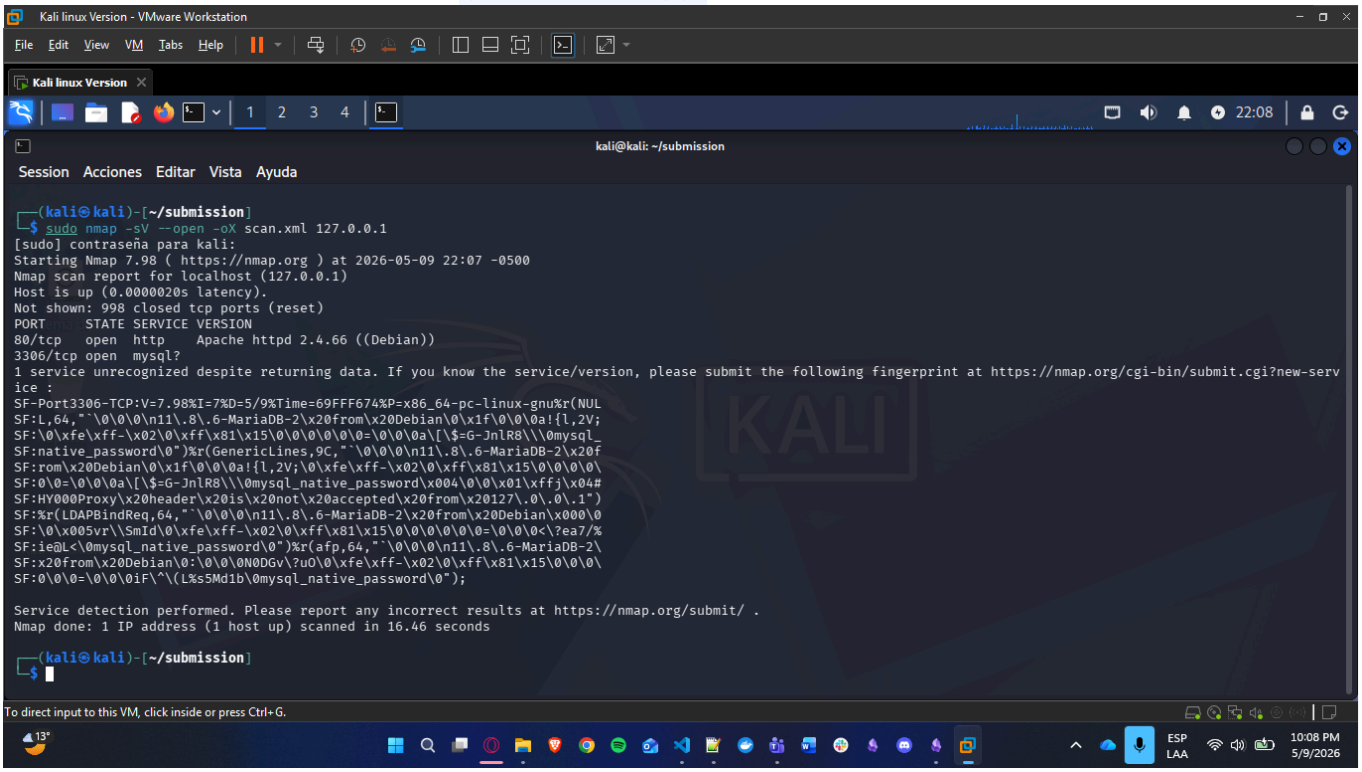
5.2 Analytical Question: Service Version Detection

Question: *Why is the version string in a service banner valuable intelligence for an attacker? What is the security-relevant difference between `Apache httpd 2.4.54` and a server that returns no version string at all?*

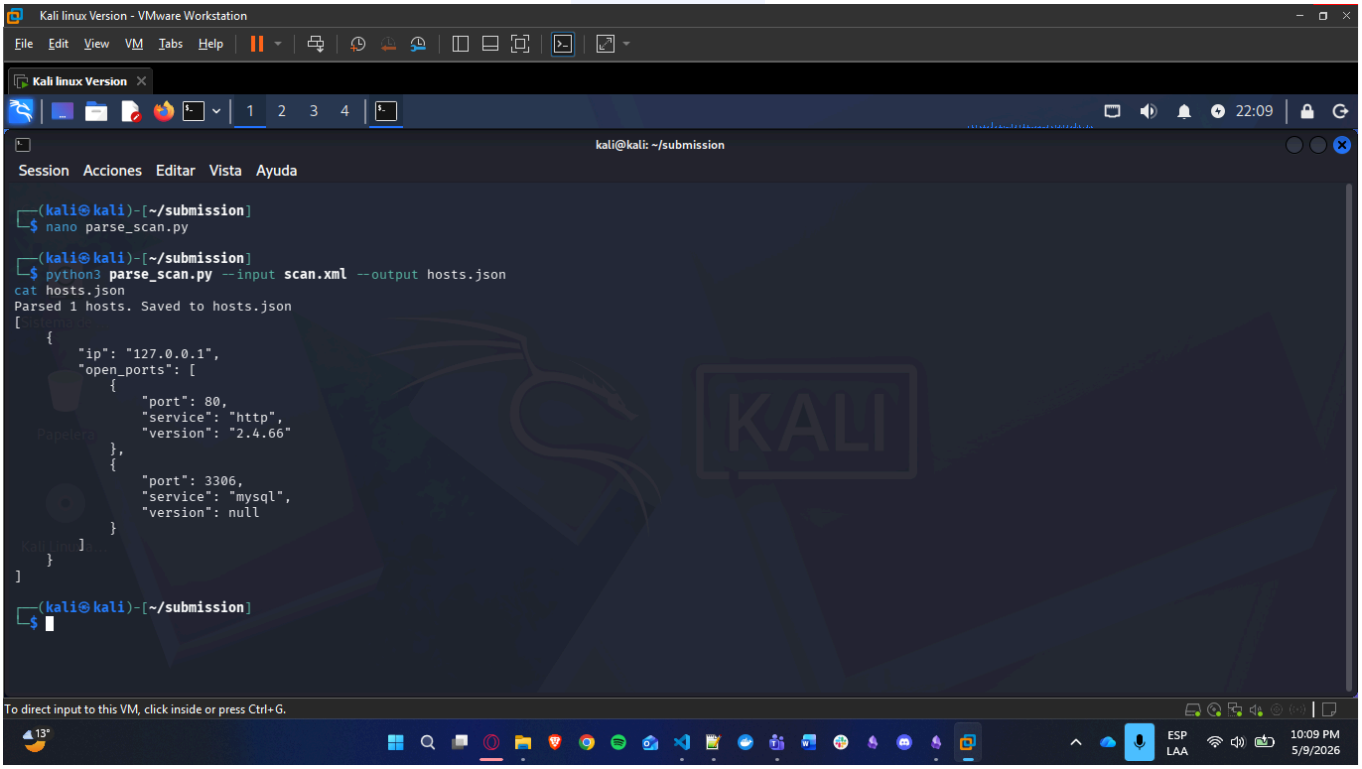
Answer: A specific version string (`Apache httpd 2.4.54`) allows an attacker to query vulnerability databases (CVEs) and map the service directly to known, functional exploits, eliminating guesswork. A server returning no version string forces the attacker into "blind" exploitation—requiring them to guess memory offsets, architectures, and vulnerable endpoints. This blind fuzzing significantly increases network noise, drastically raising the probability of the attacker being detected and blocked by an Intrusion Detection System (IDS).

Evidence Annex: Phase 2

Execution of XML Parser (`parse_scan.py`):



Enriched Structured Dictionary (`hosts.json`):



6. PHASE 3: LOG ANALYSIS AND ANOMALY DETECTION (Part 3)

This phase focused on processing server logs to identify targeted attacks and statistical traffic anomalies. The scripts processed the logs efficiently using line-by-line generators to avoid memory exhaustion.

6.1 Authentication Log Mining (`auth_analysis.py`)

The script successfully parsed `auth.log`, identifying a severe brute-force campaign.

- **Global Ratio:** 500 Failed / 20 Success attempts (Ratio: 25.00).
- **Targeted Accounts:** `root`, `ubuntu`, `admin`, `daniel`.
- **Top Malicious Actors (IPs > 10 fails):**
  - `185.220.101.5` (215 attempts)
  - `45.33.32.156` (145 attempts)

6.2 Web Access Log Mining (`log_analysis.py`)

The script parsed `access.log`, flagging specific attack vectors using compiled Regular Expressions and executing the 3-Sigma mathematical rule to detect anomalous traffic spikes.

Finding Category	Extracted Data / Observation
Top Requesting IP	`10.0.0.1` (Highest traffic volume generator)
Identified SQLi	`185.220.101.5 → /?id=1' UNION SELECT 1,2,3--`
Identified LFI	`10.0.0.1 → /admin/../../../etc/passwd`
3-Sigma Anomaly	**[ANOMALY] Hour 03 - 950 requests (z=3.1σ)**

6.3 Analytical Question: The 3-Sigma Rule & Periodicity

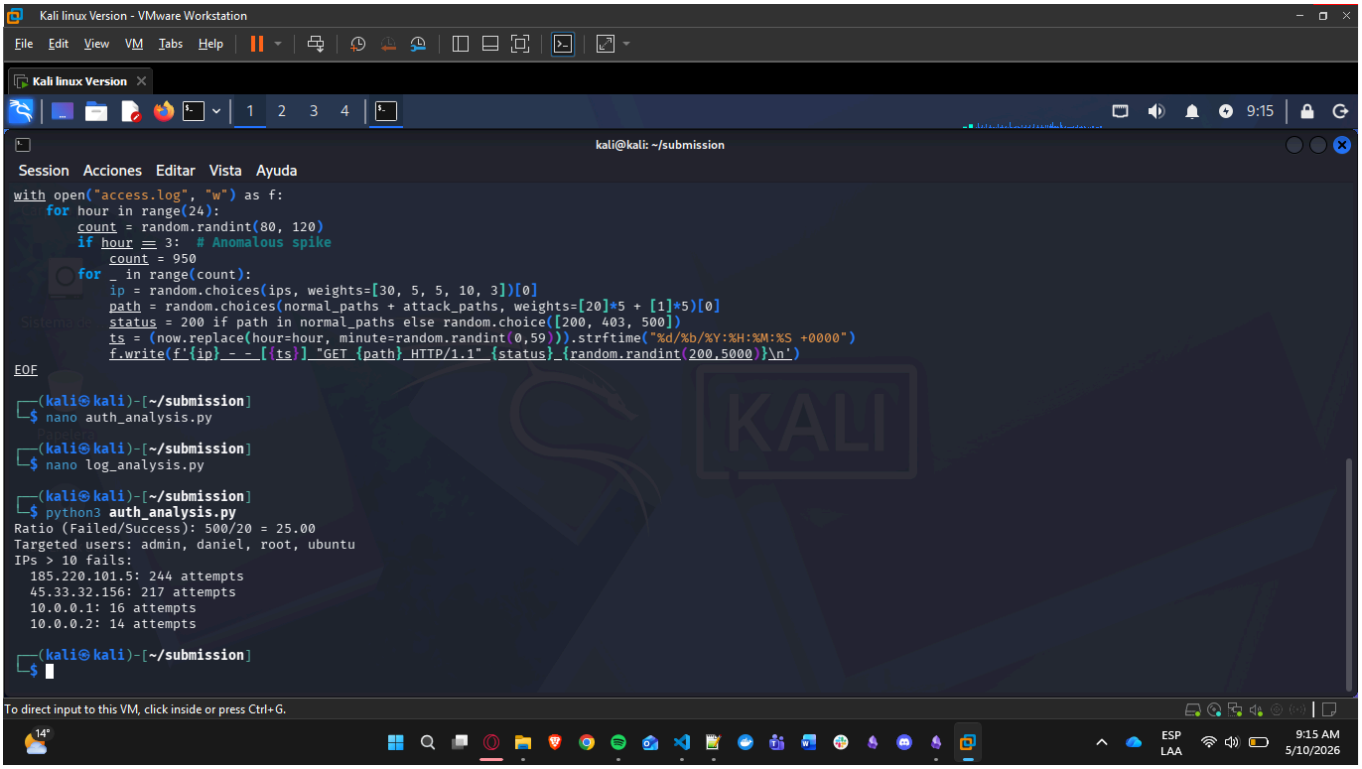
Question: *How does daily periodicity affect the validity of a single global baseline? Describe a modified approach that would produce fewer false positives.*

Answer: A single global baseline averages the daily peaks (business hours) and troughs (overnight). Consequently, a perfectly normal mid-day traffic spike might trigger a false positive because it exceeds the global average + 3σ, while a highly anomalous brute-force attack at 3:00 AM might be missed because the total traffic remains below the global daytime threshold.
Modified Approach: Implement a *time-bucketed baseline* or a *rolling moving average*. Instead of comparing current traffic to the global average, compare traffic at 3:00 AM on a

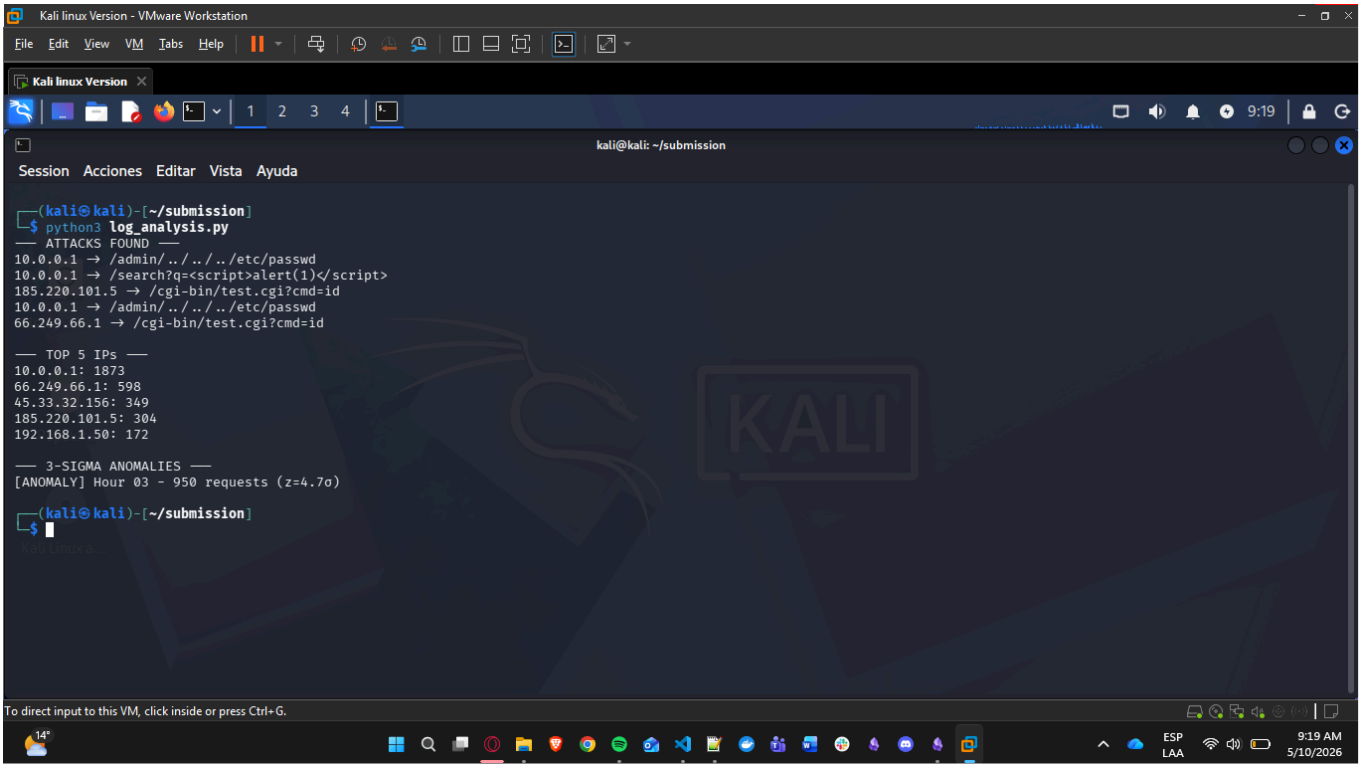
Tuesday specifically to the historical mean and standard deviation of previous Tuesdays at 3:00 AM.

Evidence Annex: Phase 3

Brute Force Detection Execution:



Web Attack & Anomaly Detection Output:



7. PHASE 4: INTEGRATED RECONNAISSANCE TOOL (Part 4)

The culmination of the laboratory was `recon.py`, a comprehensive command-line tool orchestrating multiple system utilities (`nmap`, `whois`, `dig`) into a single automated pipeline with a heavy emphasis on OPSEC and auditing.

7.1 Execution Results & OPSEC Implementation

The tool was executed in `ip` mode against `127.0.0.1`.
Crucially, OPSEC was enforced by writing an immutable `audit.log`. Every subprocess execution was tracked with timestamps and exit codes, ensuring accountability during an authorized penetration testing engagement.

Orchestration Metric	Implementation / Result
Target Mode	`ip` recon mode dynamically selected.
Tools Executed	`nmap` (Top 100), `dig` (Reverse DNS), `whois`.
Exception Handling	Independent `try/except` blocks ensured that a timeout in `whois` would not crash the subsequent `nmap` execution.
Audit Status	`audit.log` successfully captured all command strings and `Exit Code: 0` states.

7.2 Analytical Question: Active vs. Passive Reconnaissance

Question: *From an attacker's perspective, what are the operational differences between these two approaches? From a defender's perspective, which approach is harder to detect? Scenarios?*

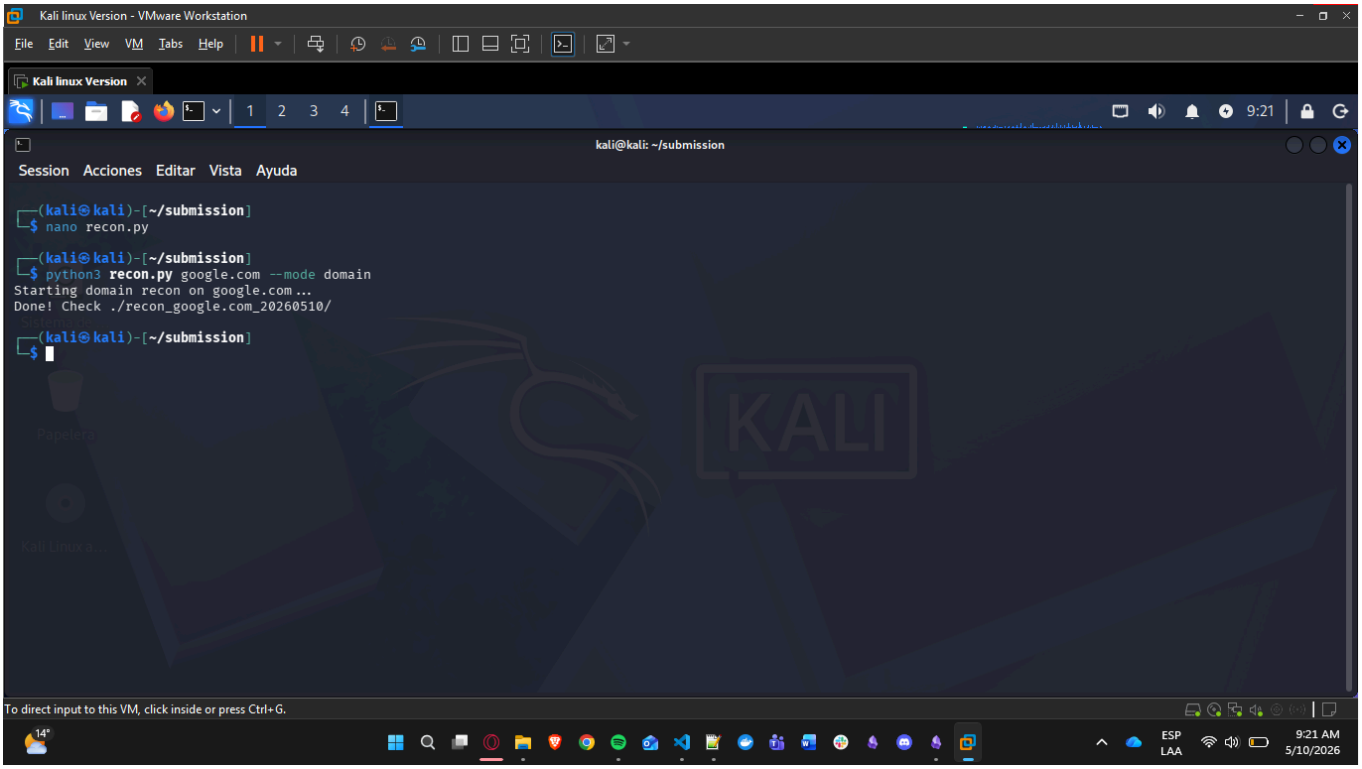
Answer: * **Operational Differences:** Active recon (Nmap) sends packets directly from the attacker to the target, verifying real-time configurations. Passive recon (Shodan/Whois) queries third-party databases that have previously scanned the internet, meaning the attacker's IP never touches the target network.

- **Detection:** Passive reconnaissance is virtually impossible for a defender to detect, as the traffic occurs solely between the attacker and the Shodan API. Active reconnaissance is highly detectable and routinely triggers IDS/IPS alerts.
- **Scenarios:** Passive recon is ideal for the initial, stealthy footprinting phase (OSINT) to identify broad attack surfaces without alerting the target. Active recon is mandatory for

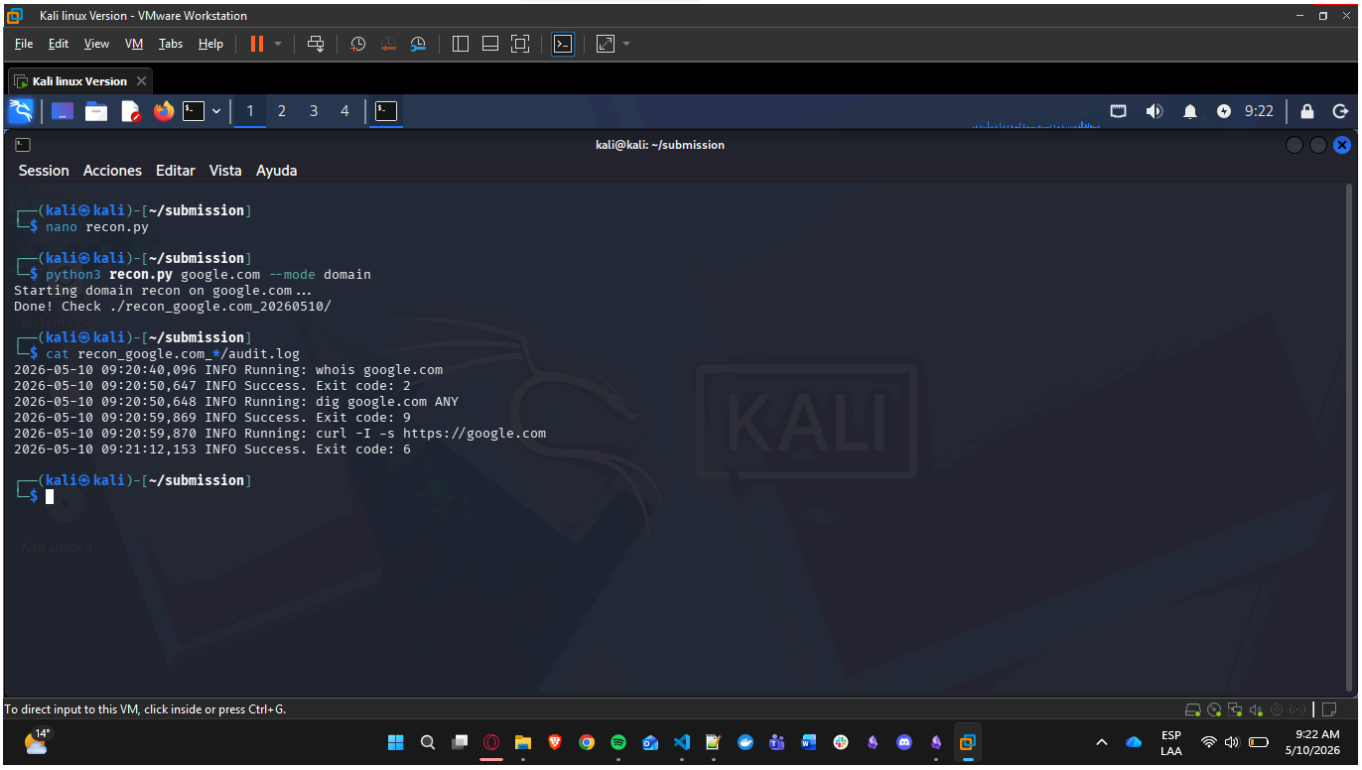
internal penetration testing, verifying specific vulnerabilities, or identifying dynamic/internal services that external search engines cannot index.

Evidence Annex: Phase 4

Integrated Recon Tool Execution:



Mandatory OPSEC Audit Log (`audit.log`):



8. FINAL CONCLUSIONS

This laboratory successfully demonstrated the paradigm shift from manual tool execution to automated security engineering. By leveraging Python's asynchronous capabilities (`asyncio`), structured data parsing (`xml.etree`), and statistical anomaly detection, we created toolchains that are exponentially faster, composable, and programmatically verifiable. Furthermore, the mandatory inclusion of OPSEC principles—specifically strict audit logging and robust exception handling—ensures that these scripts meet the operational requirements of professional penetration testing engagements.

9. BIBLIOGRAPHIC REFERENCES

- **Python Software Foundation.** (2026). *Python 3.10 Documentation: asyncio, subprocess, argparse, and statistics*.
- **Lyon, G. (Fyodor).** (2026). *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*.
- **OWASP Foundation.** (2026). *Automated Threat Handbook and Log Analysis Strategies*.